

MedArchive API — Документация

Хакатон 2025 · Med Partners · Кейс 2 (MedArchive)

Версия 1.0.0

- 1. Назначение
- 2. Архитектура и технологический стек
- 3. Базовый URL, форматы и обработка ошибок
- 4. Конвейер обработки документа
- 5. Модель данных
 - 5.1 Service — целевой справочник услуг
 - 5.2 Partner — клиника-партнёр
 - 5.3 PriceDocument — исходный прайс-документ
 - 5.4 PriceItem — позиция прайса
 - 5.5 PriceItemHistory — версии цен (хранятся бессрочно)
 - 5.6 User — учётная запись (роли)
- 6. Аутентификация и роли
- 7. Нормализация и валидация (параметры)
- 8. Справочник эндпоинтов
- 9. Эндпоинты — детально
 - 9.1 POST /api/catalog/import — импорт справочника
 - 9.2 POST /api/archives — загрузка прайсов
 - 9.3 GET /api/archives — список документов
 - 9.4 GET /api/archives/{doc_id} — статус документа
 - 9.5 GET /api/services — справочник услуг
 - 9.6 GET /api/services/{service_id}/partners — цены по услуге
 - 9.7 GET /api/partners — список партнёров
 - 9.8 GET /api/partners/{partner_id} — карточка партнёра
 - 9.9 GET /api/partners/{partner_id}/services — прайс партнёра
 - 9.10 GET /api/search?q= — полнотекстовый поиск
 - 9.11 GET /api/unmatched — несопоставленные позиции
 - 9.12 GET /api/needs-review — аномалии
 - 9.13 POST /api/match — ручное сопоставление
 - 9.14 POST /api/verify — верификация позиции
 - 9.15 GET /api/dashboard/stats — метрики
 - 9.16 POST /api/admin/login — вход оператора
- 10. Запуск и переменные окружения
- 11. Соответствие ТЗ
- Приложение А. Перечень схем для иллюстраций

1. Назначение

MedArchive API — серверная часть системы автоматической обработки архива прайс-листов клиник-партнёров. Система:

- принимает входящие документы разных форматов (PDF, DOCX, XLSX, сканы);
- извлекает структурированные данные: клиника, услуга, цена (тарифы резидент/нерезидент);
- нормализует названия услуг к единому **целевому справочнику**;
- формирует верифицированную базу партнёров, услуг и цен с историей изменений;
- предоставляет **REST API** для поиска «кто оказывает услугу и по какой цене», а также очереди ручной верификации для операторов.

Документ описывает REST API: базовый URL, аутентификацию, модель данных, конвейер обработки, полный перечень эндпоинтов с примерами запросов/ответов, правила нормализации и валидации, а также инструкции по запуску.

Интерактивная версия (Swagger UI): [GET /api/docs](#) **Машиночитаемая спецификация (OpenAPI 3.0.3):** [GET /api/openapi.json](#)

2. Архитектура и технологический стек

Слой	Технологии
Веб-фреймворк	Flask + Flask-SQLAlchemy + Flask-Migrate
Аутентификация	Flask-JWT-Extended (JWT для операторов/админа)
База данных	PostgreSQL (JSONB, полнотекстовый поиск); SQLite — fallback для быстрого старта
Очередь обработки	Celery + Redis (синхронный режим — <code>?sync=1</code> для демо)
Извлечение данных	<code>pdfplumber</code> / <code>PyMuPDF</code> (PDF-текст), <code>pytesseract</code> + Tesseract (OCR сканов), <code>python-docx</code> (DOCX), <code>openpyxl</code> (XLSX)
Нормализация	<code>rapidfuzz</code> (нечёткое сопоставление); опционально <code>sentence-transformers</code> (семантика)
CORS	разрешены <code>localhost:3000</code> (фронтенд CRA), <code>5173</code> (Vite)

Принцип: добавление нового источника/формата не требует изменений ядра — достаточно добавить экстрактор в `services/extractors/` (см. §4).

Схема для иллюстрации: «Диаграмма архитектуры» (см. Приложение А).

3. Базовый URL, форматы и обработка ошибок

- **Базовый URL:** `/api` (в dev — `http://localhost:5252/api`).
- **Формат обмена:** `application/json` (UTF-8); загрузка файлов — `multipart/form-data`.
- **Денежные значения:** число (`number`), валюта приведена к **KZT (₸)**; оригинал цены и исходная валюта сохраняются отдельно.
- **Идентификаторы:** строковые UUID (`service_id` , `partner_id` , `doc_id` , `item_id`).
- **Даты:** ISO-8601 (`YYYY-MM-DD` для дат, `...Z` для меток времени).

Проверка доступности: `GET /api` → `{"service": "MedArchive API", "status": "ok"}`

Коды ответов

Код	Значение
200 OK	Успешный запрос (чтение/действие)
201 Created	Документы приняты в обработку (<code>POST /archives</code>)
400 Bad Request	Некорректное тело/параметры запроса
401 Unauthorized	Неверный логин/пароль (<code>/admin/login</code>)
403 Forbidden	Недостаточно прав (требуется роль)
404 Not Found	Сущность не найдена
500 Internal Server Error	Внутренняя ошибка

Тело ошибки единообразно: `{ "error": "Текст сообщения" }`.

4. Конвейер обработки документа

Каждый загруженный файл проходит конвейер (`services/pipeline_service.py`). Статусы документа: `pending` → `processing` → `done` | `needs_review` | `error`.

1. **Приём (ingest).** ZIP распаковывается, для каждого файла создаётся `PriceDocument` (оригинал сохраняется в хранилище и не удаляется — ТЗ 5).
2. **Извлечение (extract).** По формату выбирается экстрактор:
 - PDF-текст — `pdfplumber` / `PyMuPDF` ; если текста нет — **автоматический fallback на OCR** (`scan_pdf` , Tesseract, языки `rus+eng+kaz`);
 - XLSX — обход листов, поиск строки заголовков, колонок «услуга»/«цена»;

- DOCX — извлечение таблиц (с принятием правок), текста.
- 3. **Валидация строки (validate).** Пустое название — строка пропускается; цена ≤ 0 — обнуляется и помечается на ревью; нерезидент < резидента — предупреждение.
- 4. **Конвертация валют.** Не-KZT приводятся к KZT по курсу на дату прайса (оригинал сохраняется). *Курсы* — заглушка `FX_RATES`; *в проде* — *API НБ РК*.
- 5. **Дедупликация / версионирование.** Если у этой клиники уже есть активная позиция с таким же названием — старая **архивируется** в `PriceItemHistory` (хранится бессрочно), новая становится активной.
- 6. **Проверка аномалий.** Отклонение цены от предыдущей версии > 50 % (`PRICE_ANOMALY_PCT`) → `has_anomaly = true` , позиция уходит в очередь ревью.
- 7. **Нормализация (normalize).** Название сопоставляется со справочником (§7). При уверенности ≥ 0.85 — авто-привязка; иначе — очередь `unmatched` .
- 8. **Итог.** Если всё нормализовано и нет предупреждений — `done` , иначе `needs_review` ; при отсутствии распознаваемых данных — `error` .

Схемы для иллюстрации: «Блок-схема конвейера обработки» и «Схема решения по нормализации» (см. Приложение А).

5. Модель данных

UUID хранятся строкой (совместимость SQLite/PostgreSQL).

5.1 Service — целевой справочник услуг

Поле	Тип	Описание
<code>service_id</code>	uuid	Идентификатор
<code>service_name</code>	string	Официальное название
<code>synonyms</code>	array	Синонимы для нормализации
<code>category</code>	string	Категория (лаборатория, диагностика, ...)
<code>icd_code</code>	string	Код по МКБ (опционально)
<code>is_active</code>	bool	Активность записи

5.2 Partner — клиника-партнёр

Поле	Тип	Описание
<code>partner_id</code>	uuid	Идентификатор
<code>name</code>	string	Название клиники
<code>city</code>	string	Город
<code>address</code>	string	Адрес
<code>bin</code>	string(12)	БИН (для дедупликации)
<code>contact_email</code> / <code>contact_phone</code>	string	Контакты
<code>is_active</code>	bool	Активность
<code>created_at</code> / <code>updated_at</code>	datetime	Метки времени

5.3 PriceDocument — исходный прайс-документ

Поле	Тип	Описание
<code>doc_id</code>	uuid	Идентификатор
<code>partner_id</code>	uuid → Partner	Клиника
<code>file_name</code>	string	Имя файла
<code>file_format</code>	enum	<code>pdf</code> / <code>docx</code> / <code>xlsx</code> / <code>scan_pdf</code>
<code>effective_date</code>	date	Дата вступления прайса в силу
<code>parsed_at</code>	datetime	Время обработки
<code>parse_status</code>	enum	<code>pending</code> / <code>processing</code> / <code>done</code> / <code>error</code> / <code>needs_review</code>
<code>parse_log</code>	text	Лог ошибок/предупреждений
<code>raw_content</code>	text	Сырой извлечённый текст (аудит, по <code>?raw=1</code>)

5.4 Priceltem — позиция прайса

Поле	Тип	Описание
item_id	uuid	Идентификатор
doc_id	uuid → PriceDocument	Исходный документ
partner_id	uuid → Partner	Клиника (денормализовано)
service_id	uuid → Service null	Нормализованная услуга
service_name_raw	string	Название как в документе
service_code_source	string	Код услуги из источника
price_resident_kzt	number	Цена резидента, KZT
price_nonresident_kzt	number	Цена нерезидента, KZT
price_original	number	Цена в исходной валюте
currency_original	enum	KZT / USD / RUB
match_score	number	Уверенность автосооставления (0..1)
is_verified	bool	Подтверждено оператором
has_anomaly	bool	Флаг аномалии цены
is_active	bool	Актуальная версия позиции
effective_date	date	Дата актуальности

5.5 PriceltemHistory — версии цен (хранятся бессрочно)

Поле	Тип	Описание
history_id	uuid	Идентификатор
item_id	uuid → Priceltem	Позиция
price_resident_kzt / price_nonresident_kzt	number	Архивные цены
effective_date	date	Дата актуальности на момент архивации
archived_at	datetime	Когда заархивировано
reason	string	Причина (новая версия / дедуп / правка)

5.6 User — учётная запись (роли)

Поле	Тип	Описание
id	int	Идентификатор
email	string	Логин
full_name	string	ФИО
role	enum	user / operator / admin
is_active	bool	Активность

Схема для иллюстрации: «ER-диаграмма базы данных» (см. Приложение А).

6. Аутентификация и роли

- **Роли:** `user` — поиск; `operator` — работа с очередями верификации; `admin` — полный доступ и аналитика.
- **Вход:** `POST /api/admin/login` возвращает **JWT** (`access_token` , срок 24 ч).
- **Передача токена:** заголовок `Authorization: Bearer <access_token>` (фронтенд добавляет автоматически).
- Публичные эндпоинты (поиск, справочник, прайсы партнёров) доступны без токена. Операторские действия (`/match` , `/verify` , очереди) предназначены для ролей `operator` / `admin` ; в ядре доступен декоратор `admin_required` для их защиты.

Демо-учётные записи (из `seed_data.py`):

Роль	Логин	Пароль
admin	<code>admin@medarchive.kz</code>	<code>admin123</code>
operator	<code>operator@medarchive.kz</code>	<code>operator123</code>
user	<code>user@medarchive.kz</code>	<code>user123</code>

Схема для иллюстрации: «Sequence-диаграмма аутентификации оператора» (см. Приложение А).

7. Нормализация и валидация (параметры)

Сопоставление со справочником (`normalization_service.py`):

1. Точное совпадение по `service_name` /синонимам → score = 1.0.
2. Нечёткий поиск (`rapidfuzz` , `token_sort_ratio`) → score 0..1.
3. (Опц.) семантический поиск по эмбедингам.

Параметр	Значение по умолчанию	Смысл
<code>MATCH_AUTO_THRESHOLD</code>	<code>0.85</code>	≥ — авто-привязка к услуге
<code>MATCH_SUGGEST_THRESHOLD</code>	<code>0.60</code>	≥ — показывать как подсказку оператору
<code>PRICE_ANOMALY_PCT</code>	<code>0.50</code>	отклонение цены > 50 % → аномалия
<code>MAX_CONTENT_LENGTH</code>	<code>500 MB</code>	макс. размер загрузки
<code>OCR_LANGS</code>	<code>rus+eng+kaz</code>	языки распознавания

Автоматические проверки при парсинге (`validation_service.py`):

Проверка	Действие при нарушении
Название услуги не пустое	Строка пропускается, запись в лог
Цена > 0 и число	Цена обнуляется, статус <code>needs_review</code>
Цена нерезидента ≥ резидента	Предупреждение, флаг ревью
Дата прайса не в будущем	Предупреждение
Дублирование (клиника+услуга)	Старая версия архивируется (versioning)
Отклонение цены > 50 %	<code>has_anomaly</code> , требует подтверждения
Валюта ≠ KZT	Конвертация по курсу на дату, оригинал сохраняется

8. Справочник эндпоинтов

#	Метод	URL	Тег	Назначение
1	POST	/api/catalog/import	catalog	Импорт справочника услуг
2	POST	/api/archives	archives	Загрузка ZIP / файлов прайсов
3	GET	/api/archives	archives	Список документов (фильтр по статусу)
4	GET	/api/archives/{doc_id}	archives	Статус обработки документа
5	GET	/api/services	services	Справочник услуг (фильтр по категории)
6	GET	/api/services/{id}/partners	services	Кто оказывает услугу и по какой цене
7	GET	/api/partners	partners	Список партнёров (фильтр город/статус)
8	GET	/api/partners/{id}	partners	Карточка партнёра
9	GET	/api/partners/{id}/services	partners	Полный прайс партнёра
10	GET	/api/search?q=	search	Полнотекстовый поиск
11	GET	/api/unmatched	review	Несопоставленные позиции
12	GET	/api/needs-review	review	Позиции с аномалиями
13	POST	/api/match	review	Ручное сопоставление позиции
14	POST	/api/verify	review	Подтвердить/отклонить/исправить
15	GET	/api/dashboard/stats	dashboard	Метрики обработки
16	POST	/api/admin/login	admin	Вход оператора/админа (JWT)

9. Эндпоинты — детально

9.1 POST /api/catalog/import — импорт справочника

Загрузка целевого справочника услуг. Принимает **JSON-массив** услуг или **XLSX-файл** (`multipart/form-data`, поле `file`; колонки `service_name`, `synonyms` через `;`, `category`, `icd_code`). Выполняется upsert.

Запрос (JSON):

```
curl -X POST http://localhost:5252/api/catalog/import \
  -H "Content-Type: application/json" \
  -d '[{"service_name": "МРТ головного мозга", "synonyms": ["магнитно-резонансная томография головы"], "category": "диагностика", "icd_code": ""}]'
```

Ответ **200** :

```
{ "created": 1, "updated": 0, "total": 1 }
```

9.2 POST /api/archives — загрузка прайсов

Принимает **ZIP-архив** (поле `file`) либо **отдельные файлы** (поле `files`). Документы ставятся в очередь обработки. Параметр `?sync=1` — синхронная обработка без Celery/Redis (удобно для демо).

Запрос:

```
# ZIP-архив, синхронная обработка
curl -X POST "http://localhost:5252/api/archives?sync=1" \
  -F "file=@prices_archive.zip"

# Отдельные файлы
curl -X POST http://localhost:5252/api/archives \
  -F "files=@Клиника1_прайс.pdf" -F "files=@Клиника2.xlsx"
```

Ответ **201** :

```
{ "source": "20260626_143000_prices_archive.zip", "documents": 8,
  "doc_ids": ["7b1f...", "9c2a...", "..."] }
```

Ошибка **400** : { "error": "Ожидается ZIP в поле 'file' или файлы в поле 'files'" }

9.3 GET /api/archives — список документов

Параметр `status` (`pending` / `processing` / `done` / `error` / `needs_review`).

```
curl "http://localhost:5252/api/archives?status=done"
```

Ответ **200** : массив `PriceDocument` (до 500, сортировка по дате убыв.).

9.4 GET /api/archives/{doc_id} — статус документа

Параметр `?raw=1` добавляет сырой извлечённый текст.

Ответ **200** :

```
{ "doc_id": "7b1f...", "partner_id": "a12...", "file_name": "Клиника1_прайс.pdf",  
  "file_format": "pdf", "effective_date": "2026-06-01", "parsed_at": "2026-06-  
    26T14:30:12Z",  
  "parse_status": "needs_review",  
  "parse_log": "Извлечено 312, автосопоставлено 281\nАномалия цены ...",  
  "items_count": 312 }
```

Ошибка 404 : { "error": "Документ не найден" }

9.5 GET /api/services — справочник услуг

Параметр `category` . Ответ 200 : массив `Service` (активные, до 1000).

```
[ { "service_id": "c0a...", "service_name": "Общий анализ крови",  
  "synonyms": ["ОАК"], "category": "лаборатория", "icd_code": null, "is_active": true } ]
```

9.6 GET /api/services/{service_id}/partners — цены по услуге

Список клиник, оказывающих услугу, **отсортированный по цене резидента**.

Ответ 200 :

```
{ "service": { "service_id": "c0a...", "service_name": "МРТ головного мозга", "category":  
  "диагностика" },  
  "partners": [  
    { "partner_id": "a12...", "partner_name": "Клиника «Альфа»", "city": "Алматы",  
      "price_resident_kzt": 14900.0, "price_nonresident_kzt": 17100.0,  
      "effective_date": "2026-06-01", "is_verified": true } ] }
```

Ошибка 404 : { "error": "Услуга не найдена" }

9.7 GET /api/partners — список партнёров

Параметры `city` , `is_active` (1 / true). Ответ 200 : массив `Partner` .

9.8 GET /api/partners/{partner_id} — карточка партнёра

Ответ 200 : объект `Partner` . Ошибка 404 : { "error": "Партнёр не найден" }

9.9 GET /api/partners/{partner_id}/services — прайс партнёра

Все активные позиции клиники с нормализованным именем услуги и категорией.

Ответ 200 :

```
{ "partner": { "partner_id": "a12...", "name": "Клиника «Альфа»", "city": "Алматы" },
  "items": [
    { "item_id": "f31...", "service_name_raw": "МРТ головн. мозга",
      "service_name": "МРТ головного мозга", "category": "диагностика",
      "price_resident_kzt": 14900.0, "price_nonresident_kzt": 17100.0,
      "currency_original": "KZT", "match_score": 0.95, "is_verified": true,
      "effective_date": "2026-06-01", "is_active": true, "has_anomaly": false } ] }
```

9.10 GET /api/search?q= — полнотекстовый поиск

Поиск по названиям услуг и партнёров (MVP — `ILIKE`).

```
curl "http://localhost:5252/api/search?q=мрт"
```

Ответ `200` : { "query": "мрт", "services": [Service...], "partners": [Partner...] }

9.11 GET /api/unmatched — несопоставленные позиции

Позиции без `service_id` + топ-подсказка из движка нормализации.

Ответ `200` :

```
[ { "item_id": "d77...", "service_name_raw": "Консульт. кардиолога",
  "price_resident_kzt": 9800.0, "service_id": null, "match_score": 0.72,
  "suggestion": { "service_id": "e90...", "service_name": "Консультация кардиолога",
    "score": 0.72 } } ]
```

9.12 GET /api/needs-review — аномалии

Ответ `200` : массив `PriceItem` с `has_anomaly = true` .

9.13 POST /api/match — ручное сопоставление

Привязать позицию к существующей услуге **или** создать новую.

Тело: { "item_id": "d77...", "service_id": "e90...", "note": "проверено" } либо { "item_id": "d77...", "new_service_name": "Новая услуга", "category": "процедура" }

Ответ `200` : обновлённый `PriceItem` (`is_verified=true` , `match_score=1.0`). Ошибки: `404` — позиция не найдена; `400` — не указан `service_id` / `new_service_name` .

9.14 POST /api/verify — верификация позиции

Тело: { "item_id": "f31...", "action": "confirm" | "reject" | "correct", "price_resident": 15000, "price_nonresident": 17200, "note": "..." }

- `confirm` — подтвердить (снять аномалию, `is_verified=true`);
- `reject` — отклонить (`is_active=false`);

- `correct` — исправить цены и подтвердить.

Ответ `200` : обновлённый `PriceItem`. Ошибки: `404`, `400` (неизвестный `action`).

9.15 GET /api/dashboard/stats — метрики

Ответ `200` :

```
{ "documents": { "total": 8, "by_status": {"done": 6, "needs_review": 2},
  "done": 6, "errors": 0, "needs_review": 2 },
  "items": { "total": 4218, "matched": 3530, "unmatched": 688,
    "anomalies": 23, "normalization_rate_pct": 83.7 },
  "partners": 12, "services": 540 }
```

`normalization_rate_pct` — доля автонормализованных позиций (цель MVP ≥ 70 %).

9.16 POST /api/admin/login — вход оператора

Тело: `{ "username": "operator@medarchive.kz", "password": "operator123" }` (поле `email` — синоним `username`).

Ответ `200` :

```
{ "access_token": "eyJhbGciOi...",
  "user": { "id": 2, "email": "operator@medarchive.kz",
    "full_name": "Оператор верификации", "role": "operator", "is_active": true } }
```

Ошибка `401` : `{ "error": "Неверный логин или пароль" }`

10. Запуск и переменные окружения

Docker (рекомендуется):

```
cp .env.example .env
docker compose up --build # postgres + redis + backend + celery worker
```

Локально (быстрый старт, SQLite-fallback):

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python seed_data.py # демо-справочник и учётки
python app.py # http://localhost:5252/api
```

Без Redis обрабатывайте загрузку синхронно: `POST /api/archives?sync=1`. Для OCR нужны

системные Tesseract (языки rus/kaz) и poppler (в Docker включены).

Ключевые переменные окружения (`.env`): `DATABASE_URL` , `REDIS_URL` , `SECRET_KEY` , `JWT_SECRET_KEY` , `MATCH_AUTO_THRESHOLD` , `PRICE_ANOMALY_PCT` , `OCR_LANGS` , `ADMIN_USERNAME` , `ADMIN_PASSWORD` , `MAX_CONTENT_LENGTH` .

11. Соответствие ТЗ

Требование ТЗ	Реализация
4.1 Загрузка и первичная обработка архива	<code>POST /api/archives</code> , очередь, сохранение оригиналов
4.2 Извлечение по форматам (PDF/скан/XLSX/DOCX)	экстракторы + OCR-fallback
4.3 Нормализация и сопоставление	<code>normalization_service</code> , порог 0.85, очередь <code>unmatched</code>
4.4 Валидация и верификация	<code>validation_service</code> , аномалии, версионирование, <code>/verify</code>
4.5 API поиска (OpenAPI)	эндпоинты §8–9, Swagger <code>/api/docs</code>
4.6 UX оператора / дашборд	<code>/unmatched</code> , <code>/needs-review</code> , <code>/dashboard/stats</code>
5. Сохранность и версионирование	оригиналы не удаляются, <code>PriceItemHistory</code> бессрочно

Приложение А. Перечень схем для иллюстраций

Следующие схемы рекомендуется добавить как рисунки (генерируются отдельно):

- Диаграмма архитектуры** — клиент (React) → Flask API → (Celery/Redis, экстракторы, нормализация) → PostgreSQL + файловое хранилище.
- Блок-схема конвейера обработки** — `extract` → `validate` → конвертация валют → дедуп/версионирование → нормализация → статус (`done`/`needs_review`/`error`).
- Схема решения по нормализации** — точное совпадение → `fuzzy` → пороги 0.85 (авто) / 0.60 (подсказка) → авто-привязка либо очередь `unmatched` .
- ER-диаграмма БД** — `Service` · `Partner` · `PriceDocument` · `PriceItem` · `PriceItemHistory` · `User` со связями.
- Sequence-диаграмма аутентификации** — оператор → `/admin/login` → JWT → защищённые операторские действия (`/match` , `/verify`).